Software Engineering Module 2 Assignment

Wenyin Zhou

August 6, 2026

1 Introduction

My research concerns generative machine learning models for biomolecular structure prediction. These models learn to generate three-dimensional structures of proteins, ligands, and protein-ligand complexes. A central application is structure-based drug discovery, where predicted structures can help researchers understand molecular interactions, identify promising drug candidates, and reduce the cost of experimental screening.

This research is not only an algorithmic problem. A useful system must generate physically plausible structures, generalize to biologically relevant cases, and communicate uncertainty to researchers who may use its predictions in subsequent experiments. Moreover, biomolecular generative models are potentially dual-use. The same capabilities that support therapeutic molecule discovery could, in principle, be misused to search for harmful or toxic molecules. Consequently, model accuracy must be considered together with validation, safety requirements, access control, human oversight, and responsible deployment. This essay discusses software engineering principles that are particularly relevant to developing such models responsibly.

2 Lecture principles

2.1 Validation beyond predictive accuracy

A key distinction from the lectures is between verification—building the system correctly—and validation—building the correct system. This distinction is especially important for biomolecular generative models. Correct implementation and strong benchmark performance do not necessarily imply that a model is useful for drug discovery. For example, an implementation may correctly optimize its training objective while producing structures that are chemically implausible, unreliable on unfamiliar protein families, or insufficiently accurate for downstream screening.

Evaluation should therefore reflect the intended scientific use rather than a single aggregate metric. In my research, this means testing geometric accuracy, physical validity, robustness to different input structures, and the reliability of confidence estimates. Evaluation should also examine important subsets of the data, such as difficult targets with substantial conformational changes. Validation connects model development to the actual needs of structural biologists and medicinal chemists.

2.2 An engineering mindset for responsible AI

The lectures present AI engineering as more than constructing and tuning a model. It also includes testing, integration, maintenance, scale, human factors, and ethics. This broader engineering mindset is important because a biomolecular model may eventually become one component of a larger experimental or drug-discovery workflow.

For a dual-use model, quality should therefore include safety and responsibility, not only predictive performance. Relevant measures could include documenting intended and prohibited uses, recording model and dataset limitations, monitoring how the system is accessed, and requiring human review before generated molecules influence laboratory decisions. These measures cannot eliminate misuse, but they make safety an explicit system property rather than an issue considered only after model development.

3 Guest lecture principles

3.1 Separate the problem space from the solution space

The requirements-engineering guest lecture distinguishes the problem space—why a system is needed and what it should accomplish—from the solution space—how it is implemented. This is relevant because it is easy to begin with a fashionable technical solution, such as diffusion or flow-matching models, without first defining the scientific problem precisely.

In my research, the problem should be expressed in terms of stakeholder needs: for example, generating plausible protein-ligand structures, supporting scientific interpretation, and reducing experimental search. Only afterwards should architectural choices be made. Safety should also appear in the problem space. Requirements may state that the system must communicate uncertainty, restrict inappropriate use, and preserve traceability. These goals should not be treated merely as optional implementation details.

3.2 Human accountability in AI-mediated work

The guest lecture on human aspects emphasizes that introducing AI changes more than code. It also changes workflows, decision rights, responsibilities, and accountability. This is particularly important in biomolecular research because a generated structure is not a scientific conclusion by itself. Researchers must decide whether it is credible, whether additional simulations or experiments are required, and whether acting on it is safe.

The model should therefore support rather than replace expert judgement. Responsibilities should be explicit: developers are responsible for documenting technical limitations, domain experts for interpreting biological plausibility, and organizations for controlling deployment and responding to misuse. Maintaining human oversight prevents responsibility from being transferred to an opaque model and makes generative biomolecular systems easier to evaluate, audit, and govern.

4 Data scientists, software engineers, and AI engineers

The CMU chapters describe data scientists as primarily concerned with data, statistical modelling, model architecture, hyperparameter selection, and evaluation on held-out datasets. Their workflow is often exploratory and science-like, commonly involving notebooks and repeated experimentation. In contrast, software engineers focus on building and maintaining complete products. Their concerns include requirements, architecture, integration, testing, deployment, scalability, security, cost, and long-term maintainability [1].

I agree that this distinction captures common differences in training and priorities, but it should not be treated as a strict boundary. My own research already combines both roles. Designing a generative model for biomolecular structures requires statistical and scientific experimentation, while training it reliably also requires data pipelines, distributed computation, testing, version control, and reproducible evaluation. The roles therefore remain useful as descriptions of expertise, but not as rigid job categories.

A system-wide view is particularly important in biomolecular structure prediction. A model with strong benchmark accuracy is not automatically a useful scientific system. Its value also depends on the quality and provenance of structural data, preprocessing and alignment procedures, computational cost, confidence estimation, physical validity, robustness to unfamiliar targets, and integration with downstream simulation or experimental workflows. Safety is likewise a system-level concern: access policies, logging, human review, and misuse monitoring cannot be implemented through the generative model alone.

I expect data scientists and software engineers to remain distinct specializations, while both acquire broader knowledge of the other field. Data scientists need to understand production constraints and maintainable pipelines, whereas software engineers working with learned components need to understand data distributions, statistical evaluation, and model uncertainty. The most effective collaboration will therefore involve T-shaped researchers: deep in one area but able to communicate across the complete system.

AI engineering lies at this intersection but is partly distinct. It combines systematic model development, software engineering, MLOps, data engineering, quality assurance, and responsible deployment. It is not merely conventional software engineering applied to an AI component, because learned behaviour, data dependence, uncertainty, drift, and difficult-to-specify outputs create additional engineering problems.

Foundation models and AI coding assistants further blur these boundaries. Engineers can construct useful systems through prompts, retrieval-augmented generation, tool use, and agentic workflows without training the underlying model. At the same time, the work shifts toward evaluating model behaviour, designing context and orchestration pipelines, monitoring failures, and defining human oversight. AI reduces some implementation barriers, but it increases the need for system integration, evaluation, governance, and clear accountability.

5 Paper analysis

5.1 Paper 1: Technical debt in machine-learning systems

5.1.1 Core ideas and software-engineering importance

The code-related technical debt is investigated across the machine-learning life cycle [3]. Starting from 34 candidate issues and refining them through focus groups with experienced ML professionals, they identify 30 relevant issues, 24 of which were judged highly important. Data preprocessing was the most critical stage, including shortcuts in handling missing values, outliers, inconsistencies, scaling, rebalancing, and feature selection.

The central software-engineering lesson is that technical debt is not confined to the final model implementation. Temporary scripts, duplicated transformations, implicit assumptions, and "patch fixes" throughout the data and training pipeline can make results difficult to reproduce and systems expensive to modify. This is important because ML research often rewards rapid experimental progress while hiding future maintenance costs.

5.1.2 Relation to my research

This problem is directly visible in biomolecular structure prediction. A training pipeline may include structure filtering, atom and residue mapping, coordinate alignment, graph construction, conformer generation, sampling, relaxation, and several evaluation procedures. Small undocumented differences between these stages can change the reported performance. Experimental scripts that initially support one dataset or model can consequently become fragile dependencies as the project grows.

5.1.3 Integration into a larger AI-intensive project

Consider a drug-discovery platform that predicts flexible protein–ligand structures, ranks generated poses, and forwards selected candidates to simulation and experimental screening. The paper's issue catalogue could be used as a review checklist for its data, training, and evaluation repositories. Preprocessing steps would be modular and versioned; intermediate datasets would carry provenance metadata; transformations would be tested; and training configurations, model checkpoints, and evaluation results would be traceable.

My research would provide the platform's generative docking component. Its integration would require an explicit interface specifying molecular inputs, generated structures, confidence values, and known failure conditions rather than treating the model as an isolated research script.

5.1.4 Adaptation of my research

I would adapt the project by treating the full experimental pipeline as a maintained software product. Shared preprocessing modules would replace dataset-specific patches, tests would check geometric invariants and atom mapping, and continuous evaluation would detect changes caused by code or data updates. This requires some initial effort but reduces the risk that apparent model improvements originate from untracked pipeline differences.

5.2 Paper 2: Measuring and reducing malicious capabilities

5.2.1 Core ideas and software-engineering importance

Li et al. introduce the Weapons of Mass Destruction Proxy (WMDP) benchmark for evaluating hazardous knowledge in biological, chemical, and cybersecurity domains [2]. The benchmark contains 3,668 expert-written multiple-choice questions that were filtered to avoid releasing sensitive information. The authors also introduce representation misdirection for unlearning, which reduces performance on hazardous knowledge while attempting to preserve general capabilities.

Its main engineering contribution is to make misuse risk measurable. Safety requirements such as "the system should not facilitate biological harm" are otherwise too abstract to test. WMDP demonstrates how threat modeling, domain-specific evaluations, and mitigation testing can become part of the quality-assurance process. Nevertheless, benchmark performance remains a proxy and cannot establish that a deployed system is safe.

5.2.2 Relation to my research

Although WMDP evaluates language models rather than molecular generators, its threat model is highly relevant. A generative structure model can support drug discovery, but similar capabilities might help optimize harmful compounds or biomolecular interactions. Conventional docking metrics measure structural accuracy and physical validity, but they do not measure whether increased capability also increases misuse potential.

5.2.3 Integration into a larger AI-intensive project

In the proposed drug discovery platform, safety evaluation would form a separate layer alongside scientific validation. Before deployment, red-team scenarios would test whether the combined language interface, structure generator, search tools, and external databases enable prohibited workflows. Access controls, query and generation logging, rate limits, and expert review could be activated according to the estimated risk. This system-level approach is necessary because a harmless-looking component may become dangerous when connected to other tools.

My generative docking research would contribute useful structural capabilities, while the safety layer would govern how those capabilities are exposed. The scientific model and its deployment policy would thus be evaluated jointly.

5.2.4 Adaptation of my research

I would extend my evaluation beyond prediction quality by developing domain-specific dual-use test cases with biosecurity and toxicology experts. The test suite should measure both hazardous capability and retained beneficial utility, reflecting the trade-off studied in WMDP. I would also design the software interface to support controlled access and auditable generation rather than releasing only unrestricted model weights. This would make safety a research objective and an architectural requirement rather than a final deployment decision.

6 Evidence, originality, and GenAI-use transparency

The main sources used in this essay were the course lecture slides, the two assigned chapters of *Machine Learning in Production*, and the two papers analysed in Section 5.

The course slides by Robert Feldt and the guest lecturers were teaching materials rather than peer-reviewed research papers. I used them to identify the software-engineering principles discussed in Sections 2 and 3, including validation, requirements engineering, human accountability, and the need for a system-wide view of AI development.

The chapters from *Machine Learning in Production* were used as book chapters. They provided the main basis for the discussion of the differences between data scientists, software engineers, and AI engineers, and for the distinction between model-centric and system-centric development [1].

The paper by Ximenes et al. is a peer-reviewed CAIN conference paper. I used it to discuss technical debt across machine-learning pipelines, especially in data preprocessing and experimental infrastructure [3]. The paper by Li et al. is a peer-reviewed ICML conference paper. I used it to discuss measurable dual-use risk, safety evaluation, and mitigation of hazardous model capabilities [2].

I place the greatest trust in the peer-reviewed papers because their methods and claims have undergone formal academic review. I also trust the CMU book chapters as a carefully developed academic source, although they are broader and more instructional in character. The lecture slides were useful for understanding the course framing, but I treated them mainly as guidance and interpretation rather than as primary empirical evidence.

Generative AI was used to help organize the essay and improve wording. I also checked the cited sources and remained responsible for the accuracy and originality of the submitted text.

References

[1] Christian Kästner and Eunsuk Kang. *Machine Learning in Production: From Models to Products*. Carnegie Mellon University, 2025. Chapters 1 and 2, accessed 2026-08-06.

[2] Nathaniel Li, Alexander Pan, Anjali Gopal, Summer Yue, Daniel Berrios, Alice Gatti, Justin D. Li, Ann-Kathrin Dombrowski, Shashwat Goel, Gabriel Mukobi, Nathan Helm-Burger, Rassin Lababidi, Lennart Justen, Andrew B. Liu, Michael Chen, Isabelle Barrass, Oliver Zhang, Xiaoyuan Zhu, Rishub Tamirisa, Bhrugu Bharathi, Ariel Herbert-Voss, Cort B. Breuer, Andy Zou, Mantas Mazeika, Zifan Wang, Palash Oswal, Weiran Lin, Adam A. Hunt, Justin Tienken-Harder, Kevin Y. Shih, Kemper Talley, John Guan, Ian Steneker, David Campbell, Brad Jokubaitis, Steven Basart, Stephen Fitz, Ponnurangam Kumaraguru, Kallol Krishna Karmakar, Uday Kiran Tupakula, Vijay Varadharajan, Yan Shoshitaishvili, Jimmy Ba, Kevin M. Esvelt, Alexandr Wang, and Dan Hendrycks. The wmdp benchmark: Measuring and reducing malicious use with unlearning. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 28525–28550. PMLR, 2024.

[3] Rodrigo Ximenes, Antonio Pedro Santos Alves, Tatiana Escovedo, Rodrigo Spinola, and Marcos Kalinowski. Investigating issues that lead to code technical debt in machine learning systems. In *Proceedings of the 2025 IEEE/ACM 4th International Conference on AI Engineering–Software Engineering for AI (CAIN)*, pages 173–183. IEEE, 2025.